[*pattern*, *pattern*, ...]

[**variable*, *pattern*, *pattern*, ..., **variable*]

An array pattern matches if every element of the array expression matches, so [Integer, Integer] matches [1, 2], but not [1]. But you can use the splat character as a wild card to match more than one element at the beginning or ending of an array pattern. If you use a splat at both the beginning and the end that's called a *find pattern*

```
case [1, 2, 3]
in [Integer, Integer, Integer]
  puts "all integers"
in [*, Integer, *]
  puts "contains an integer"
else
  puts "no integers"
end
```

produces:

```
all integers
```

{*key*: *pattern*, *key*: *pattern*, ..., ‹**nil›+}

Hash patterns behave differently from array patterns. A hash pattern will match if all the keys in the pattern match the target expression, even if there are additional keys in the target expression. Appending **nil to the end of the pattern will change this behavior, then the pattern will only match if the keys in the pattern are the only keys in the target expression.

```
case {first_name: "Ron", last_name: "Lithgow"}
in {first_name: String}
  puts "this person has a first name"
else
  puts "this data does not have a first name"
end
```

produces:

```
this person has a first name
```

The empty hash is an exception, it only matches other empty hashes.

pattern | *pattern*

The pipe character (|) can be used to specify multiple alternative patterns that might match, Ruby will bind to the first of the multiples that matches.

Binding Variables in Patterns

pattern => *variable*

At any point in a pattern, you can bind a subpattern to a local variable by appending => VAR_NAME to the sub pattern:

```
case [1, 2, 3]
in [Integer, Integer => middle, Integer]
  puts "all integers, the middle one is #{middle}"
in [*, Integer, *]
  puts "contains an integer"
```